



Unifor

Problema B — CSES Labyrinth

Resolução de Problemas com Grafos — Trabalho Prático 1

Alunos:

Leonardo Enzo Pinheiro Diógenes - 2420372

Ester dos Santos da Silva - 2324328

Orientador: Prof. Me. Ricardo Carubbi

Disciplina: Resolução de Problemas com Grafos

Centro de Ciências Tecnológicas - CCT

Universidade de Fortaleza - UNIFOR



Problema B — CSES Labyrinth

Apresentação

Agenda

- Problema (1 min);
- Modelagem (1 min);
- Representação computacional (1 min);
- DFS/BFS (1 min);
- Validação (1 min).



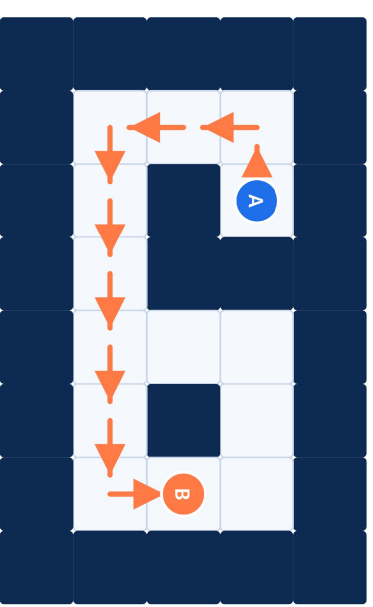
Problema

CSES 1193 — *Labyrinth*

O que é pedido

- Mapa $n \times m$ com paredes (#), piso (.), início (A) e fim (B);
- Mover apenas para cima, baixo, esquerda e direita;
- Encontrar o caminho mais curto de A até B;
- Saída: “YES”, o comprimento e a sequência L/R/U/D — ou “NO”.

Exemplo (5x8)



Saída: YES / 9

LDDRRRRRU

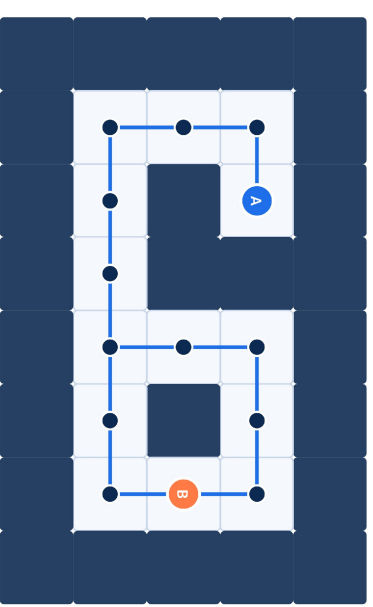
Modelagem

Grafo, vértices, arestas e instância pequena

Grafo implícito da grade

- Vértices: cada célula transitável (., A ou B);
- Arestas: entre células vizinhas nas 4 direções, ambas livres;
- Tipo: grafo não direcionado e não ponderado;
- Instância pequena: exemplo do enunciado (5×8);
- Hipótese: BFS a partir de A encontra o caminho mais curto até B.

Grafo da grade



Vértices (círculos) e arestas (linhas) sobre o exemplo 5×8.

Representação computacional

Leitura da entrada e construção do grafo

Estrutura de dados

- Representação implícita: matriz de caracteres `grade[n][m]`;
- Leitura da entrada: armazenar cada linha do mapa e localizar A e B;
- Vizinhança gerada sob demanda com os 4 deltas $(-1,0)$, $(1,0)$, $(0,-1)$, $(0,1)$ — sem lista de adjacência explícita;
- Medidas estruturais: $|V| \leq n \cdot m$ células livres; grau máximo 4; $|E| \leq 4 \cdot |V|$;
- Validação: aplicar a construção à instância pequena e conferir manualmente as vizinhanças geradas.

DFS/BFS

Execução manual e escolha justificada

BFS a partir de A

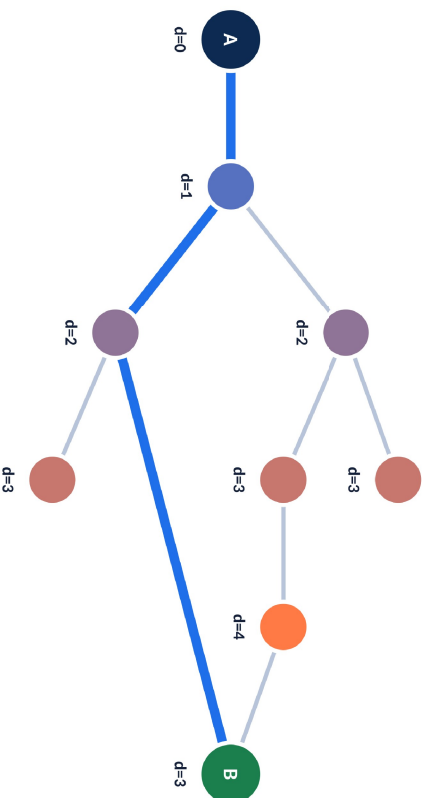
- Fila com A; $\text{dist}[A] = 0$ e demais células com $\text{dist} = -1$ (não visitado);
- Ao expandir uma célula, visita-se os vizinhos livres não marcados: $\text{dist} = \text{dist}(\text{atual}) + 1$, guarda-se apenas a posição do predecessor;
- Busca encerra ao alcançar B (ou quando a fila esvazia $\rightarrow$ sem caminho);
- Caminho: percorre-se os predecessores de B até A, converte-se cada passo em D/U/R/L pela diferença de coordenadas e inverte-se a sequência;
- Por que BFS e não DFS: o grafo não é ponderado — o nível da BFS é exatamente a distância, garantindo o caminho mais curto; a DFS não oferece essa garantia;
- Complexidade: $O(n \cdot m)$, dentro do limite de tempo de 1 s.

BFS vs. DFS

Comparação visual · mesmo grafo, duas ordens de busca

BFS — busca em largura

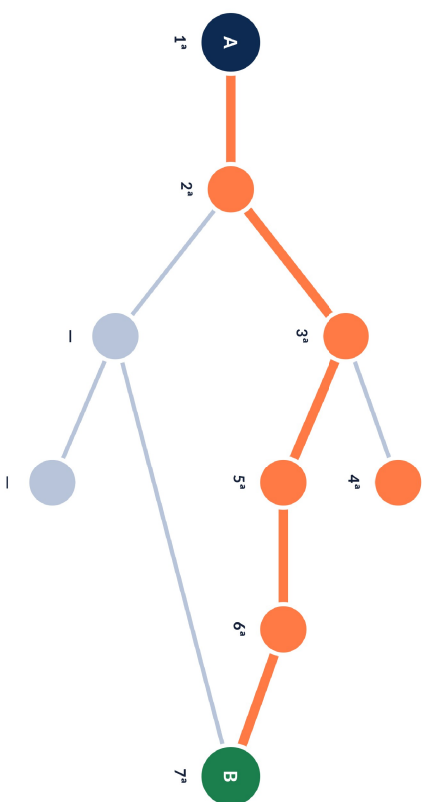
explora por camadas de distância



Caminho encontrado: A → n1 → n2 → B (3 arestas — o mais curto)

DFS — busca em profundidade

explora um ramo até o fim antes de voltar



Caminho encontrado: A → n1 → n3 → n4 → n5 → B (5 arestas — não é o mais curto)

A BFS avança por camadas de distância e garante o caminho mais curto; a DFS aprofunda um ramo antes de voltar, sem essa garantia.

Validação

Adaptação, testes e Accepted · 1 minuto

Testes e submissão

- Instância do enunciado (5×8) → saída esperada “YES”, comprimento 9 e um caminho válido (ex.: LDDRRRRRU);
- Casos de borda: sem caminho possível → “NO”; labirinto mínimo com A e B adjacentes;
- Adaptação da implementação de referência: guardar apenas a posição do predecessor (edge_to) e derivar a direção do movimento pela diferença de coordenadas ao reconstruir o caminho;
- Reprodutibilidade: script de execução e casos de teste versionados no repositório;
- Evidência: print do veredito “Accepted” no juiz (CSES) anexado em evidencias/.

Adaptação do código

Da BFS da aula ao grafo implícito da matriz

BFS adaptada ao grafo implícito

- Mantivemos a estrutura do BreadthFirstPaths visto em aula;
- Como o grafo é implícito na matriz, o método `adj(v)` substitui `G.adj(v)` e verifica as quatro células vizinhas;
- `_marked` passou a ser uma matriz booleana e `edge_to` guarda a posição predecessora de cada célula;
- O restante da lógica é o mesmo: marca a origem, insere na fila, retira o primeiro elemento e adiciona os vizinhos ainda não visitados;
- Quando B é alcançado, usamos `edge_to` para voltar de B até A;
- Invertamos esse caminho e transformamos as diferenças entre as coordenadas em D, U, R e L.



Unifor